

Planejamento do Projeto: Interpretador de Comandos Orientado a Objetos (Mini-Shell)

1. Visão Geral

Este projeto desafia a equipe a construir um interpretador de comandos interativo (*shell*) orientado a objetos. O objetivo não é apenas ler texto e invocar ordens do sistema operacional, mas criar uma arquitetura extensível capaz de gerenciar o ciclo de vida de rotinas, interpretar sintaxes com metacaracteres, manipular redirecionamentos de entrada e saída, controlar variáveis de ambiente e gerenciar sinais do sistema.

2. Objetivos de Aprendizado e Aplicação (POO)

1. Polimorfismo e Extensibilidade (Padrão Command):

- Tratar cada instrução (comandos internos, programas externos, pipelines) como uma abstração derivada de uma classe base ou interface comum. O interpretador processará requisições polimorficamente.

2. Encapsulamento e Proteção do Estado:

- Assegurar que os dados da aplicação (como tabelas de variáveis de ambiente e histórico de comandos) fiquem ocultos do escopo global. Qualquer manipulação do estado interno deve ocorrer estritamente por meio de métodos seletores e modificadores protegidos (*getters* e *setters*).

3. Gerenciamento de Recursos e Resiliência (Tratamento de Exceções):

- Implementar tratamento robusto de erros para falhas de sintaxe, arquivos inacessíveis ou interrupções de execução. O encerramento de um comando filho falho não pode derrubar a aplicação principal. Todo recurso alocado (como descritores de arquivos abertos e memória associada) deve ser liberado de forma determinística após o uso.

4. Concorrência e Assincronismo:

- Utilizar abstrações de concorrência e processos para manipular múltiplos comandos conectados via canais de comunicação em cadeia (*pipes*) sem causar deadlocks ou concorrência desordenada.

3. Requisitos Funcionais do Sistema

Módulo Core (Obrigatório)

- **Prompt e Histórico:** Exibição contínua de um indicador visual de prontidão para aguardar entrada e manutenção de um registro dos comandos inseridos.
- **Análise Sintática (*Parser*):**
 - Suporte ao isolamento de texto via aspas simples ' (inibe a interpretação de todos os metacaracteres) e aspas duplas " (inibe metacaracteres, exceto expansões).
 - Expansão de variáveis de ambiente (ex.: `$VARIABEL`) e do estado de saída da última rotina executada (`$?`).
- **Redirecionamentos e Cadeias de Comunicação:**
 - Redirecionamento de entrada (<) e saída em modo sobrescrita (>) ou anexação (>>).
 - Suporte a blocos de entrada textual contínua baseados em delimitador (*heredoc* <<).
 - Conexão da saída de um comando à entrada do seguinte utilizando cadeias de comunicação (*pipe* |).
- **Instruções Internas (*Built-ins*):**
 - Implementação direta na própria aplicação para: navegação de diretórios, exibição de texto, listagem do diretório atual, definição/remoção de variáveis de ambiente, listagem do ambiente e encerramento controlado.
- **Tratamento de Sinais:**
 - A aplicação interativa deve capturar e tratar sinais de interrupção do sistema (como `ctrl-C`, `ctrl-D`, `ctrl-\`) garantindo que o programa principal não seja encerrado indevidamente. Apenas a indicação do sinal pode ser repassada de forma isolada sem violar o encapsulamento das estruturas internas.

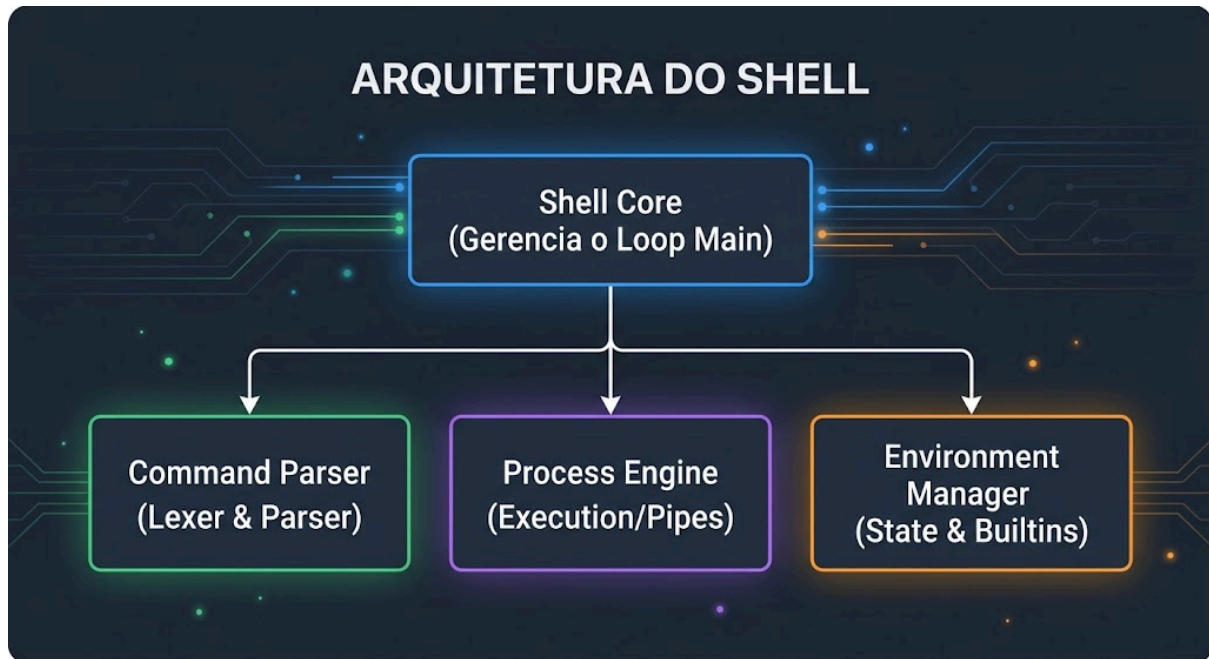
Módulo Bônus (Opcional)

- **Operadores Lógicos e Precedência:** Suporte a operadores de execução condicional (`&&` e `||`) controlados por parênteses para agrupamento.
- **Expansão de Padrões (*Wildcards*):** Suporte ao caractere `*` para correspondência de arquivos no diretório atual.

4. Arquitetura OO e Divisão de Responsabilidades (Equipe de 2 Alunos)

Para garantir a entrega no prazo estipulado, o trabalho deve ser estruturado nos seguintes papéis e componentes:

ARQUITETURA DO SHELL



1. Parser & Lexer Engineer:

- **Papel:** Processar a cadeia de caracteres digitada pelo usuário, gerando uma árvore de sintaxe abstrata ou uma fila de objetos executáveis.
- **Responsabilidades:** Ocultar a complexidade de remoção de aspas, isolar metacaracteres e validar erros sintáticos antes da execução.

2. Process Engine Specialist:

- **Papel:** Gerenciar a criação de subprocessos, redirecionamentos de arquivo e encadeamento via canais de comunicação.
- **Responsabilidades:** Implementar a execução dos objetos de comandos externos, gerenciar concorrência e tratar a sincronização do encerramento das rotinas.

3. Environment & Built-ins Manager:

- **Papel:** Encapsular as variáveis do sistema, o histórico de comandos e a execução dos comandos internos (*built-ins*).
- **Responsabilidades:** Implementar as classes dos comandos internos utilizando o padrão *Command* e garantir que as alterações no ambiente sejam propagadas corretamente para a aplicação.

4. Signal & Resource Sentinel:

- **Papel:** Proteger a integridade da aplicação contra vazamentos de memória, descritores de arquivos pendentes e interrupções por sinais inesperados.
- **Responsabilidades:** Implementar mecanismos globais de captura de sinais sem violar o encapsulamento, assegurando a liberação determinística de recursos ao final do ciclo de cada execução.

5. Entregas e Critérios de Avaliação

1. Demonstração Funcional:

- Apresentar a aplicação rodando em modo interativo.
- Demonstrar o encadeamento de múltiplos comandos com *pipes*, redirecionamento para arquivos de disco e uso das aspas.

2. Análise do Projeto de Software:

- Apresentação do Diagrama de Classes detalhando a aplicação dos conceitos de Herança, Polimorfismo, Encapsulamento e Separação de Responsabilidades.
- Verificação de resiliência: a aplicação não deve travar nem encerrar de forma inesperada diante de entradas inválidas ou falhas de comandos externos.

3. Documentação (*README*):

- Elaboração de documentação contendo a descrição técnica das decisões de projeto, instruções claras para compilação/construção e execução, e detalhamento de recursos de apoio e uso de ferramentas de suporte.